Somaiya Vidyavihar University

(A Constituent College of Somaiya Vidyavihar University)

Batch: A-4	**Roll No.:** 16010122151
Experiment / Assignment / Tutorial No: 7	

Title: Implementation of input modeling steps for simulation

Objective: To understand and implement the process of developing input models for simulation by following the four essential steps: data collection, identifying probability distributions, choosing parameters for the distributions, and evaluating the goodness of fit. The experiment aims to highlight the significance of high-quality input data in producing reliable simulation outputs.

Expected Outcome of Experiment:

CO4: Analyze the systems for input modeling and validation.

CO5: Estimate the different parameters of absolute and relative performance of different simulation systems.

Books/ Journals/ Websites referred:

1. "Discrete-Event System Simulation" by Jerry Banks, John S. Carson II, Barry L. Nelson, David M. Nicol.
2. SimPy Documentation: https://simpy.readthedocs.io/en/latest/
3. SciPy Documentation: https://docs.scipy.org/doc/scipy/

Background:

(Explain in brief steps for input modeling development in simulation.)

Problem Statement:

Perform the following steps of Input Model Development:

1. **Collect Data from the Real System:**
   - Identify the key input processes in the system (e.g., arrival times, service times).
   - Collect data through direct observation, historical records, or sensors.
   - Ensure data is recorded accurately and is sufficient in quantity to perform statistical analysis.

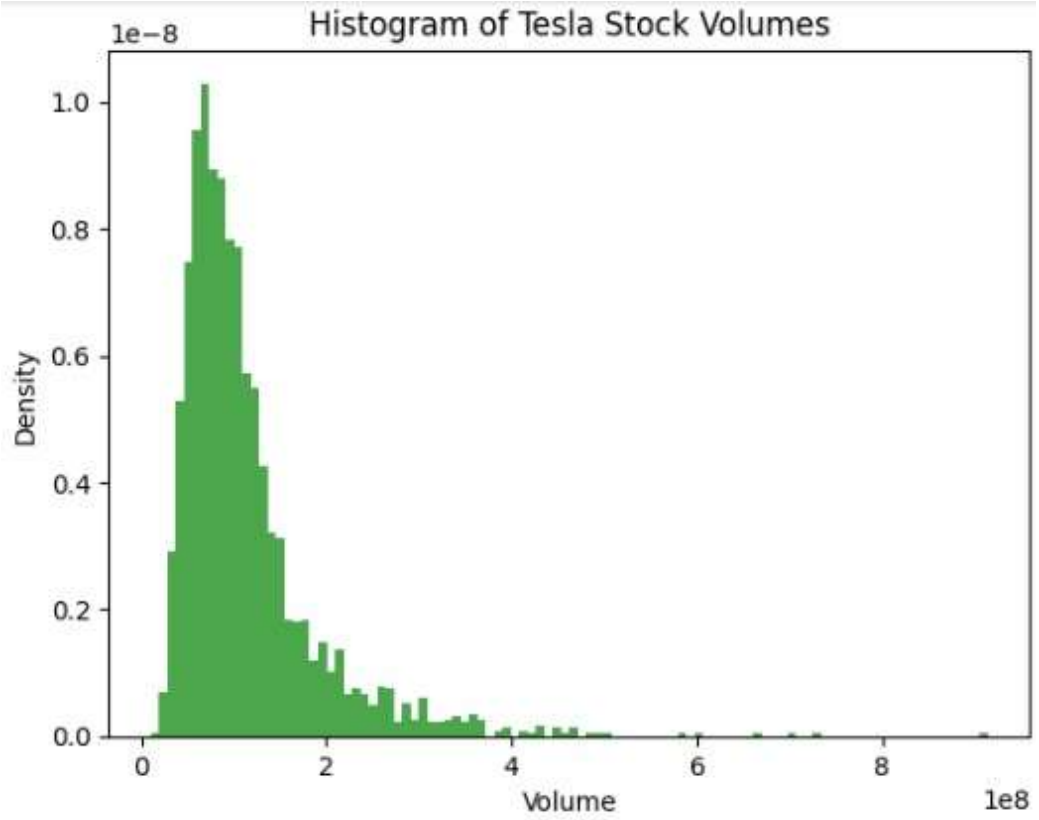
2. **Identify a Probability Distribution to Represent the Input Process:**
   - Plot the collected data to visually inspect its distribution (e.g., using histograms).
   - Use statistical software or tools to fit different probability distributions to the data.
   - Compare the fit of different distributions using statistical measures (e.g., likelihood, AIC).

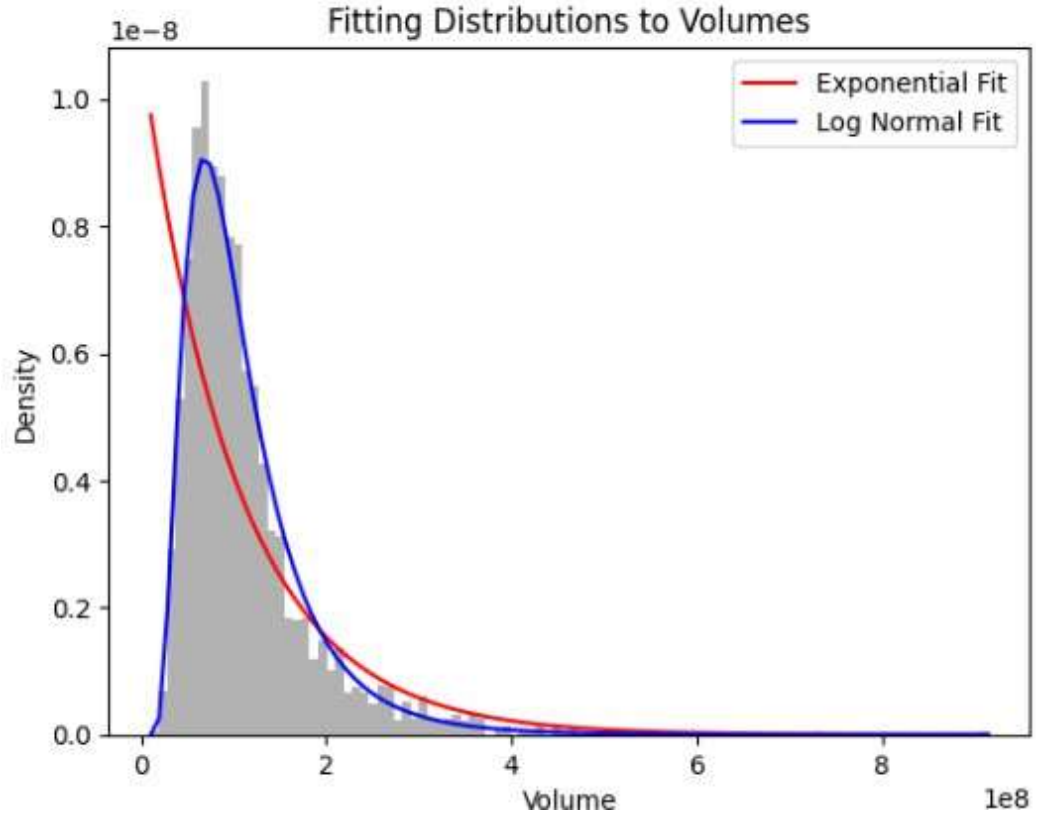
3. **Choose Parameters for the Distribution:**
   - Use statistical software or tools to estimate the parameters of the chosen distribution.
   - Ensure that the estimated parameters make sense in the context of the real system.

4. **Evaluate the Chosen Distribution and Parameters for Goodness of Fit:**
   - Perform goodness-of-fit tests such as the Kolmogorov-Smirnov test, Chi-square test.
   - Compare the observed data with the expected values from the chosen distribution.

Implementation Steps with Screen shots:

CODE : Attached with submission


Histogram of Tesla Stock Volumes
1e−8
1.0
0.8
0.6
0.4
0.2
0.0
Density
0
2
4
6
8
Volume
1e8


Fitting Distributions to Volumes
1e−8
Exponential Fit
Log Normal Fit
1.0
0.8
0.6
0.4
0.2
0.0
Density
0
2
4
6
8
Volume
1e8

```
Exponential Fit Parameters (loc, scale): (10620000.0, 102578599.20508744)
Log-Normal Fit Parameters (shape, loc, scale): (np.float64(0.5986118999925562), 7327288.557954236, np.float64(87775141.30054982))
Kolmogorov-Smirnov test for Exponential: KstestResult(statistic=np.float64(0.21853297505643754), pvalue=np.float64(5.540307966133598e-106), statistic_location=np.int64(49744500), statistic_sign=np.int8(-1))
Kolmogorov-Smirnov test for Log Normal: KstestResult(statistic=np.float64(0.029637001037754573), pvalue=np.float64(0.023589481479335293), statistic_location=np.int64(122659500), statistic_sign=np.int8(1))
Chi-square test for Exponential: Power_divergenceResult(statistic=np.float64(1123.743858306693), pvalue=np.float64(2.7590469624684107e-173))
Chi-square test for Log-Normal: Power_divergenceResult(statistic=np.float64(279.45653187637146), pvalue=np.float64(4.0540048923567216e-19))
```

Conclusion:

The log-normal distribution fits Tesla's stock volumes better than the exponential distribution, shown by lower KS and chi-square statistics. Although not perfect-small p-values show some mismatch-it models the data more accurately, while the exponential distribution performs poorly.

Post lab Questions:

1. **Explore the concept of multivariate input models. How would you approach input modeling if the input data involved multiple correlated variables?**

A multivariate input model describes the behavior of two or more random variables simultaneously. Unlike univariate modeling, which analyzes one variable in isolation, multivariate modeling focuses on the **joint probability distribution** of the variables.

The critical component is capturing the **dependence structure** or **correlation** between the variables.

For example, modeling a person's *height* is a univariate problem. Modeling their *height and weight* is a multivariate problem. You must model not only the distribution of heights and the distribution of weights but also the *relationship* that taller people tend to be heavier. A good multivariate model will generate realistic pairs of (height, weight) and avoid simulating unrealistic pairs (e.g., 7 feet tall, 100 pounds).

Approach for Modeling Correlated Variables

If I were to model input data with multiple correlated variables, I would follow a structured, multi-step process. The modern and most flexible approach separates the modeling of the individual variables' distributions from the modeling of their dependence structure.

1. Exploratory Data Analysis (EDA)

First, you must understand the nature of the variables and their relationships.

- **Visualize Marginals:** Plot a histogram for each variable individually to get a preliminary idea of its shape (e.g., symmetric, skewed, heavy-tailed).
- **Visualize Relationships:** Use a **scatterplot matrix** (also called a pair plot). This is the most important visualization. It shows histograms of each variable on the diagonal and scatterplots for every pair of variables on the off-diagonal. This immediately reveals linear, non-linear, or complex relationships.
- **Quantify Correlation:** Calculate a **correlation matrix**.
  - **Pearson's ρ(rho):** Measures the *linear* relationship. A value of 0 doesn't mean no relationship, only no *linear* one.
  - **Spearman's ρ or Kendall's τ (tau):** These are rank-based and measure *monotonic* relationships (i.e., as one variable increases, the other generally increases or decreases, even if not linearly). This is often more robust.

2. Model the Marginal Distributions

This step is a univariate problem, identical to the one in your previous code. For *each* variable, you must find its best-fit probability distribution.

1. **Select Candidate Distributions:** Based on the histograms (e.g., Log-Normal or Exponential for skewed data, Normal for symmetric data, Weibull for failure times).
2. **Fit Parameters:** Use statistical methods (like Maximum Likelihood Estimation, as `scipy.stats.fit` does) to find the parameters for each candidate distribution (e.g., `loc` and `scale`).
3. **Test Goodness-of-Fit:** Use tests like the **Kolmogorov-Smirnov (KS)** or **Chi-Square test** to statistically determine which distribution is the best fit for that specific variable.

At the end of this step, you will have a fitted distribution for X1 (e.g., Log-Normal), X2 (e.g., Exponential), X3 (e.g., Normal), and so on.

3. Model the Dependence Structure (Copulas)

This is the core of the multivariate problem. How do you "glue" these different marginal distributions together using the correlation structure you observed in Step 1?

The classic approach, using a **Multivariate Normal Distribution**, is often a poor choice because it *forces* all marginal distributions to be Normal, which is rarely true.

The superior and modern solution is to use **Copulas**.

What is a Copula?

A copula is a function that links marginal probability distributions together to form a joint distribution. The key idea (from **Sklar's Theorem**) is that any joint distribution can be *decomposed* into two parts:

1. The individual marginal distributions (which you found in Step 2).
2. The copula, which describes their dependence structure.

This is powerful because it lets you model the marginals (e.g., Log-Normal) and the dependence (e.g., strong correlation in a downturn) *separately*.

The Copula Modeling Process:

1. **Transform Data:** Convert your original data (which has different distributions) into a uniform (0, 1) scale. This is done by applying the cumulative distribution function (CDF) of each variable's fitted marginal.
   - u_1 = F_{X1}(x1) (where F_{X1} is the CDF of the fitted Log-Normal)
   - u2 = F_{X2}(x2) (where F_{X2} is the CDF of the fitted Exponential)
   - This transformed dataset, (u_1, u_2, …), now captures *only* the dependence structure.
2. **Fit a Copula:** Select and fit a copula function to this transformed u-data. Common families include:
   - **Gaussian (Normal) Copula:** The simplest. It uses a standard correlation matrix (like Pearson's ρ). It's good at modeling linear dependence but bad at "tail dependence."
   - **t-Copula:** A generalization of the Gaussian copula. It has an extra parameter for "degrees of freedom" and is excellent at modeling **tail dependence** (i.e., the tendency for variables to crash together in extreme events). This is very popular in finance.
   - **Clayton or Gumbel Copulas:** These are *asymmetric* copulas. For example, a Clayton copula can model a situation where variables are highly correlated during *downturns* (lower-tail dependence) but independent during *upturns*.
3. **Select the Best Copula:** Use information criteria like **AIC** or **BIC** to select the copula family that best fits your u-data.

4. Validation and Simulation

Your final model consists of the set of marginal distributions (from Step 2) and the chosen copula (from Step 3).

To validate, you use this model to **generate new, synthetic data**:

1. **Sample from the Copula:** Draw a new set of correlated (u_1, u_2, …) samples from your fitted copula (e.g., the *t*-copula).
2. **Inverse Transform:** Convert these uniform *u*-samples back to their original scale by using the *inverse CDF* (also called the *quantile function* or ppf in SciPy) of each marginal distribution.
   - $x1_{new} = F^{-1}_{X1}(u1)$
   - $x2_{new} = F^{-1}_{X2}(u2)$

The resulting synthetic dataset ($x1_{new}$, , $x2_{new}$ ...) should statistically resemble your original data. You can validate it by checking if its scatterplot matrix and correlation matrix look similar to the original data.